TP1 : PHP

Module : Outils de developpement | **Filière** : 3IIR

Réalisé par : Abir NASSIF | **Année universitaire** : 2026/2027

1. Introduction

Ce TP a pour objectif de manipuler les bases du langage PHP : instructions conditionnelles, boucles, fonctions, tableaux et expressions régulières. Quatre exercices sont traités : la gestion d'un tableau associatif de notes, les opérations sur les matrices, la conception d'une fonction réutilisable de génération de tableaux HTML, puis la validation de chaînes de caractères par expressions régulières.

2. Objectifs pédagogiques

- Créer, parcourir, modifier, trier et rechercher dans des tableaux associatifs.
- Manipuler des tableaux à deux dimensions (matrices) avec des boucles imbriquées.
- Factoriser du code dans une fonction et le partager entre plusieurs pages avec `require_once`.
- Valider des données avec `preg_match()` et les expressions régulières.

3. Environnement de travail

Élément	Description
Langage	PHP
Serveur local	serveur intégré `php -S localhost:8000`
Éditeur	Visual Studio Code
Organisation	Un dossier par exercice : `exercice1` à `exercice4`

4. Exercice 1 : Tableau associatif des notes

4.1 Énoncé

À partir d'un tableau `$notes` (clé = prénom, valeur = note), réaliser huit opérations : ajout, suppression, maximum et minimum, tri alphabétique, recherche d'une note, classement par mérite et moyenne de la classe. Le tableau est initialisé dans `test.php` et affiché dans `home.php`.

4.2 Démarche

home.php inclut test.php grâce à require_once, puis applique chaque question. Pour conserver le tableau modifié par les questions 2 et 3, les tris des questions 5 et 7 sont réalisés sur des **copies** du tableau.

4.3 Code principal

```
// test.php : initialisation
$notes = ["said" => 13, "badr" => 16, "najat" => 15];

// home.php
require_once 'test.php';
$notes["Ali"] = 10;                    // Q2 : ajout
unset($notes["najat"]);                // Q3 : suppression
$noteMax = max($notes);                // Q4 : maximum
$noteMin = min($notes);                //      minimum
$triAlpha = $notes;
ksort($triAlpha, SORT_STRING | SORT_FLAG_CASE); // Q5 : tri alphabétique
$presente = in_array(20, $notes, true);   // Q6 : recherche de 20
$merite = $notes;
arsort($merite);                       // Q7 : ordre de mérite
$moyenne = array_sum($notes) / count($notes); // Q8 : moyenne
```

4.4 Résultats attendus

Question	Opération	Résultat
Q2	Ajout de Ali (10)	said 13, badr 16, najat 15, Ali 10
Q3	Suppression de najat	said 13, badr 16, Ali 10
Q4	Maximum / minimum	16 (badr) / 10 (Ali)
Q5	Tri alphabétique	Ali, badr, said
Q6	La note 20 existe-t-elle ?	Absente
Q7	Classement par mérite	badr 16, said 13, Ali 10
Q8	Moyenne de la classe	13 / 20

4.5 Fonctions PHP utilisées

Fonction	Rôle
unset()	Supprime un élément à partir de sa clé

Fonction	Rôle
max() / min()	Valeur maximale / minimale
array_keys($t, $v)	Retrouve la ou les clés associées à une valeur
ksort()	Trie selon les clés (ordre alphabétique)
arsort()	Trie selon les valeurs, ordre décroissant, clés conservées
in_array()	Teste la présence d'une valeur
array_sum() / count()	Somme des valeurs / nombre d'éléments

5. Exercice 2 : Matrices

5.1 Énoncé

Écrire un script qui calcule la somme de deux matrices carrées, puis un script qui calcule le produit de deux matrices.

5.2 Principe

- **Somme :** C[i][j] = A[i][j] + B[i][j] ; les deux matrices doivent avoir les mêmes dimensions.
- **Produit :** C[i][j] = Σ A[i][k] × B[k][j] ; le nombre de colonnes de A doit être égal au nombre de lignes de B. Trois boucles imbriquées sont nécessaires.

5.3 Code principal

```
function sommeMatrices(array $A, array $B): array {
   $C = [];
   for ($i = 0; $i < count($A); $i++)
      for ($j = 0; $j < count($A[0]); $j++)
         $C[$i][$j] = $A[$i][$j] + $B[$i][$j];
   return $C;
}

function produitMatrices(array $A, array $B): array {
   $C = [];
   for ($i = 0; $i < count($A); $i++)
      for ($j = 0; $j < count($B[0]); $j++) {
         $C[$i][$j] = 0;
         for ($k = 0; $k < count($A[0]); $k++)
            $C[$i][$j] += $A[$i][$k] * $B[$k][$j];
```

```
    }
  return $C;
}
```

5.4 Résultats attendus

Avec A = [[1,2,3],[4,5,6],[7,8,9]] et B = [[9,8,7],[6,5,4],[3,2,1]] :

A + B			A × B		
10	10	10	30	24	18
10	10	10	84	69	54
10	10	10	138	114	90

6. Exercice 3 : Fonction `generateTable()`

6.1 Énoncé

Écrire dans `config.php` une fonction `generateTable()` qui génère un tableau HTML, puis l'utiliser dans `page1.php` (Name / Age) et `page2.php` (Email / Phone) selon les paramètres transmis.

6.2 Démarche

La fonction reçoit les données (tableau associatif) et les titres des colonnes, puis **retourne** le code HTML sous forme de chaîne. Cette conception évite de dupliquer le code : les deux pages se contentent d'inclure `config.php` et d'appeler la fonction avec des paramètres différents.

6.3 Code principal

```
// config.php
function generateTable(array $donnees, array $entetes): string {
   $html = "<table><thead><tr>";
   foreach ($entetes as $titre) $html .= "<th>" . htmlspecialchars($titre) . "</th>";
   $html .= "</tr></thead><tbody>";
   foreach ($donnees as $cle => $valeur)
      $html .= "<tr><td>" . htmlspecialchars($cle) . "</td><td>" . htmlspecialchars($valeur) .
"</td></tr>";
   return $html . "</tbody></table>";
}

// page1.php
require_once 'config.php';
$table1 = ['Amine' => '30', 'Ali' => '60'];
```

```
echo generateTable($table1, ['Name', 'Age']);

// page2.php
require_once 'config.php';
$table2 = ['sam@gmail.com' => '+2124444444', 'cris@gmail.com' => '+2128888888'];
echo generateTable($table2, ['Email', 'Phone']);
```

6.4 Remarques

- `htmlspecialchars()` protège l'affichage contre l'injection de code HTML (faille XSS).
- L'énoncé nomme les deux tableaux `$table1` ; le second a été renommé `$table2` pour éviter toute confusion.

7. Exercice 4 : Expressions régulières

7.1 Énoncé

Tester si une chaîne ne contient que des lettres, si elle contient le caractère `@`, puis vérifier à l'aide d'expressions régulières un numéro de téléphone (`00-00-00-00-00`), un matricule (3 lettres au début, 2 chiffres à la fin) et un code postal (5 chiffres).

7.2 Code principal

```
function contientQueDesLettres(string $s): bool { return preg_match('/^\p{L}+$/u', $s) === 1; }
function contientArobase(string $s): bool       { return strpos($s, '@') !== false; }
function estTelephone(string $s): bool          { return preg_match('/^\d{2}(-\d{2}){4}$/D', $s) === 1; }
function estMatricule(string $s): bool          { return preg_match('/^[a-zA-Z]{3}.*\d{2}$/D', $s) === 1; }
function estCodePostal(string $s): bool         { return preg_match('/^\d{5}$/D', $s) === 1; }
```

7.3 Explication des motifs

Motif	Signification
`^` et `$`	Début et fin de chaîne (la chaîne entière doit correspondre)
`\p{L}+`	Une ou plusieurs lettres (accents compris, avec le modificateur `u`)
`\d{2}`	Exactement 2 chiffres
`(-\d{2}){4}`	Le groupe « tiret + 2 chiffres » répété 4 fois
`[a-zA-Z]{3}`	Exactement 3 lettres
`.*`	Zéro ou plusieurs caractères quelconques

Motif	Signification
D	Empêche d'accepter un saut de ligne final

7.4 Jeux d'essai et résultats attendus

Test	Chaîne	Résultat
Que des lettres	Bonjour	Valide
Que des lettres	abc123	Invalide
Contient @	sam@gmail.com	Valide
Contient @	samgmail.com	Invalide
Téléphone	06-12-34-56-78	Valide
Téléphone	0612345678	Invalide
Matricule	abc12	Valide
Matricule	ab123	Invalide
Code postal	75001	Valide
Code postal	750011	Invalide

8. Difficultés rencontrées et solutions

- **Ordre des opérations (exercice 1) :** les questions modifient le même tableau ; les tris sont donc faits sur des copies pour ne pas altérer les résultats suivants.
- **Comparaison de `strpos()` :** il faut utiliser `!== false`, car une position `0` est interprétée comme faux avec un test simple.
- **Matricule :** l'énoncé peut être lu de deux façons (longueur libre ou exactement 5 caractères) ; les deux motifs sont documentés dans le code.

9. Conclusion

Ce TP m'a permis de consolider la manipulation des tableaux associatifs et multidimensionnels, la structuration du code en fonctions réutilisables et l'emploi des expressions régulières pour valider des saisies. Ces notions constituent une base indispensable pour la suite du module, en particulier le traitement des formulaires et l'interaction avec une base de données.